

Foundations of Evaluation in NLP

Yurii Laba, UCU NLP Course

Why do we evaluate model's performance?

Why do we evaluate model's performance?

- We need it because we cannot improve what we cannot measure.

Why do we evaluate model's performance?

- We need it because we cannot improve what we cannot measure.

Okay, but what does it mean to actually “**evaluate**” the model?

Why do we evaluate model's performance?

- We need it because we cannot improve what we cannot measure.

Okay, but what does it mean to actually “**evaluate**” the model?

- It means that we want to measure how well an NLP system performs relative to a standard of correctness.

Why do we evaluate model's performance?

- We need it because we cannot improve what we cannot measure.

Okay, but what does it mean to actually “**evaluate**” the model?

- It means that we want to measure how well an NLP system performs relative to a standard of correctness.

Yes, It's like grading a student's exam answers against the official answer key.

Key questions we aim to address when evaluating model.

1. How accurate is the system?
2. Is it useful in real tasks?
3. Does it generalize beyond the training data?

Key questions we aim to address when evaluating model.

1. How accurate is the system?
2. Is it useful in real tasks?
3. Does it generalize beyond the training data?

Question	Meaning	Example Task
Accuracy	Is the model right compared to gold data?	Spam detection (email → spam/ham)
Usefulness	Does the system help achieve the real-world goal?	Chatbot for password reset
Generalization	Can it handle new, unseen, or different data?	Sentiment classifier with slang

Intrinsic vs Extrinsic Evaluation

Intrinsic Evaluation

Intrinsic evaluation measures the quality of an NLP model by testing it on well-defined tasks or benchmarks that directly reflect the model's internal performance. Typical examples include:

1. language modeling
2. part-of-speech tagging
3. sentiment analysis
4. machine translation.

Intrinsic Evaluation - Pros

1. **Task-Focused:** Provides precise insights into model performance on a specific NLP task.
2. **Fast Feedback:** Results are produced quickly, supporting rapid experimentation and iteration.
3. **Benchmarking:** Uses established datasets and benchmarks.
4. **Clear Metrics:** Relies on accuracy, precision, recall, F1-score, and similar metrics for detailed assessment.
5. **Controlled Setup:** Evaluation conditions can be carefully designed and repeated.

Intrinsic Evaluation - Cons

1. **Limited Scope:** Performance on isolated tasks may not translate to real-world applications.
2. **Weak Predictive Power:** High intrinsic scores do not guarantee success in broader contexts.
3. **Task Dependence:** Outcomes depend heavily on the chosen evaluation task, limiting generalization.
4. **Disconnect from Practice:** Ignores how models function in multi-task or real-world environments.

Intrinsic Evaluation

Extrinsic evaluation measures how well an NLP model performs when applied to real-world tasks or applications. Instead of focusing on isolated benchmarks, it evaluates the model's contribution to broader objectives:

1. enhancing customer service chatbots
2. improving search engine results
3. supporting medical translation in healthcare settings

Extrinsic Evaluation - Pros

1. **Big-picture view:** Assesses how the model performs when integrated into a full system, not just in isolation.
2. **Robustness check:** Shows whether the model can handle varied, real-world situations.
3. **Practical relevance:** Tells us how much the model actually improves real applications.
4. **User focus:** Emphasizes outcomes that matter to end users, like task completion and satisfaction.
5. **Handles complexity:** Captures performance in multi-step or multi-task environments where different components interact.

Extrinsic Evaluation - Cons

1. **Resource-heavy:** Requires more time, data, and effort to design and run compared to intrinsic tests.
2. **Potentially subjective:** Often involves human evaluation, which can introduce bias or inconsistency.
3. **Hard to isolate effects:** Difficult to measure exactly how much of the outcome is due to the NLP model versus other parts of the system.
4. **Application-dependent:** Results may vary widely depending on the specific task or system in which the model is deployed.

Use case

A hospital wants to use machine translation to help doctors communicate with patients who don't speak the local language. How would you setup the evaluation pipeline?

What else do you need to evaluate your NLP model?

1. Holdout (Train/Test split)
2. Cross-validation
3. Pre Defined Benchmark

Pre Build Benchmarks Example

- Text Classification & Sentiment Analysis
 - a. IMDB Reviews (binary sentiment: positive vs negative). Example:
 - i. “The movie was a masterpiece.” → Positive
 - ii. “This was the worst film I’ve ever seen.” → Negative

- Named Entity Recognition (NER)
 - b. CoNLL-2003 (English NER: Person, Organization, Location, Misc). Example:
 - i. “Barack Obama was born in Hawaii.”
 - 1. Barack Obama → PERSON
 - 2. Hawaii → LOCATION

Pre Build Benchmarks Example

- Machine Translation
 - a. WMT (Workshop on Machine Translation) datasets - yearly benchmark with multiple language pairs.
 - i. Example: EN $\rightarrow$ DE:
 - 1. Source: “He went home”
 - 2. Reference: “Er ging nach Hause”

- SQuAD (Stanford Question Answering Dataset)
 - a. Context: “Barack Obama was the 44th president of the US.”
 - i. Q: “Who was the 44th president of the US?”
 - ii. A: “Barack Obama”

Gold Standards & Human Annotation **VS**
Semi-Supervised

Okay, finally lets discuss metrics!

Okay, finally lets discuss metrics!

One more quick warm-up question: is an evaluation metric the same as a loss function?

Text classification (binary classification)

Text classification (binary classification)

1. Classification accuracy
2. Precision
3. Recall
4. F1 score
5. ROC/AUC
6. Precision/Recall AUC
7. Matthews correlation coefficient
8. Log loss

Classification accuracy

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Classification accuracy

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
model = RandomForestClassifier(  
    n_estimators=100,  
    max_depth=None,  
    random_state=42,  
    n_jobs=-1  
)
```

```
model.fit(X_train, y_train)
```

```
score = model.score(X_test, y_test)  
print("RandomForestClassifier accuracy:", score)
```

```
RandomForestClassifier accuracy: 0.9572222222222222
```

Classification accuracy

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

```
dummy = DummyClassifier(strategy='most_frequent', random_state=42)
dummy.fit(X_train, y_train)
```

```
score = dummy.score(X_test, y_test)
print("DummyClassifier accuracy:", score)
```

```
🔗 DummyClassifier accuracy: 0.8938888888888888
```

Why is this happening?

Classification accuracy

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

```
dummy = DummyClassifier(strategy='most_frequent', random_state=42)
dummy.fit(X_train, y_train)
```

```
score = dummy.score(X_test, y_test)
print("DummyClassifier accuracy:", score)
```

```
🔗 DummyClassifier accuracy: 0.8938888888888888
```

```
X, y = make_classification(
    n_samples=30000,
    n_classes=2,
    weights=[0.90, 0.10], # 95% of one class, 5% of the other
    random_state=42
)
```

Confusion matrix

How can we tell where the model went wrong?

Confusion matrix

How can we tell where the model went wrong?

1. **Not a standalone metric:** A confusion matrix is not itself a performance score, but a structured summary of prediction outcomes.
2. **Reveals types of errors:** It breaks down predictions into true positives, true negatives, false positives, and false negatives, helping us understand exactly where the model is making mistakes.
3. **Foundation for other metrics:** Many important evaluation measures—such as accuracy, precision, recall, F1-score, and specificity—are directly derived from the confusion matrix.
4. **Provides deeper insight:** Beyond overall accuracy, it highlights class-level imbalances, showing if the model is biased toward the majority class or consistently missing minority cases.

Confusion matrix

```
y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
[[7935  110]
 [ 275  680]]
```

Confusion matrix

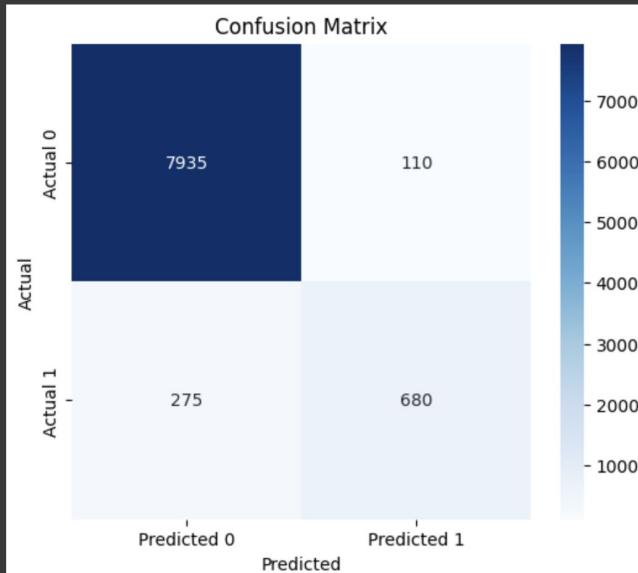
```
y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
[[7935  110]
 [ 275  680]]
```



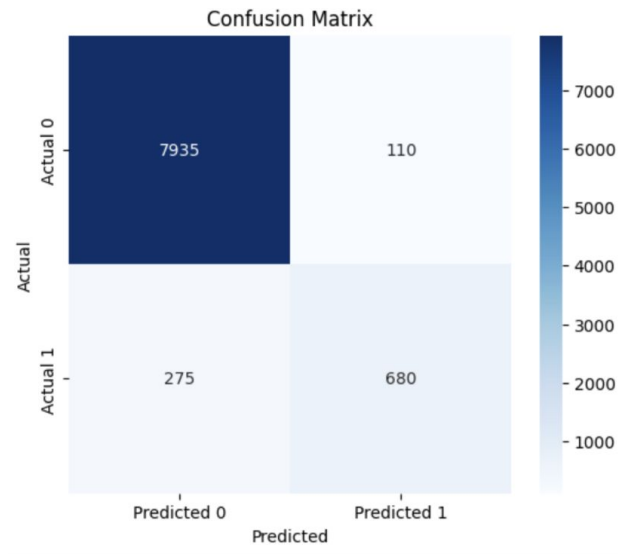
```
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["Predicted 0", "Predicted 1"],
            yticklabels=["Actual 0", "Actual 1"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Precision, Recall, F1 Score

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

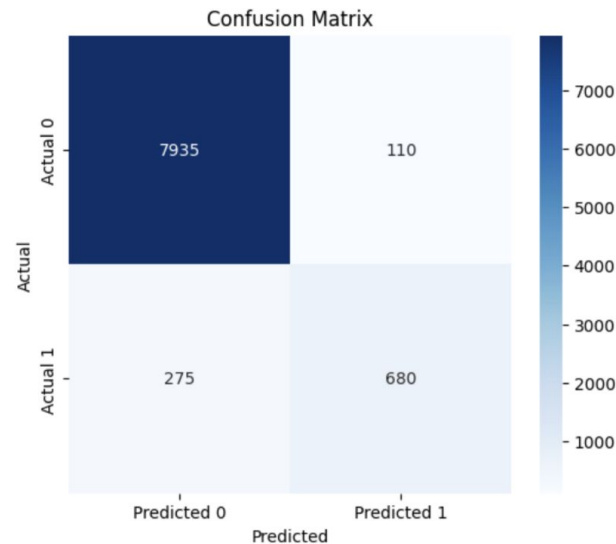


Precision, Recall, F1 Score

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1} = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$



When should you focus on precision, and
when should you focus on recall?

Metric as the objective

```
param_grid = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [None, 10, 20],  
    'min_samples_split': [2, 5, 10]  
}  
  
grid_search = GridSearchCV(  
    estimator=model,  
    param_grid=param_grid,  
    scoring='recall',  
    cv=5,  
    n_jobs=-1  
)  
  
grid_search.fit(X_train, y_train)
```

Matthews Correlation Coefficient (MCC)

Another way to summarize the confusion matrix: Unlike accuracy or precision, which only focus on certain aspects of the confusion matrix, MCC incorporates all four categories (TP, TN, FP, FN).

Balanced measure: MCC is especially useful for imbalanced datasets, since it does not get artificially inflated by a dominant majority class.

Interpretation:

+1: Perfect prediction (all labels correct).

0: No better than random guessing.

-1: Total disagreement between predictions and actual labels.

MCC vs F1 Score

	Predicted: 0	Predicted: 1
True: 0	TN = 0	FP = 10
True: 1	FN = 0	TP = 90

Lets calculate F1 and MCC!

MCC vs F1 Score

	Predicted: 0	Predicted: 1
True: 0	TN = 3	FP = 2
True: 1	FN = 4	TP = 91

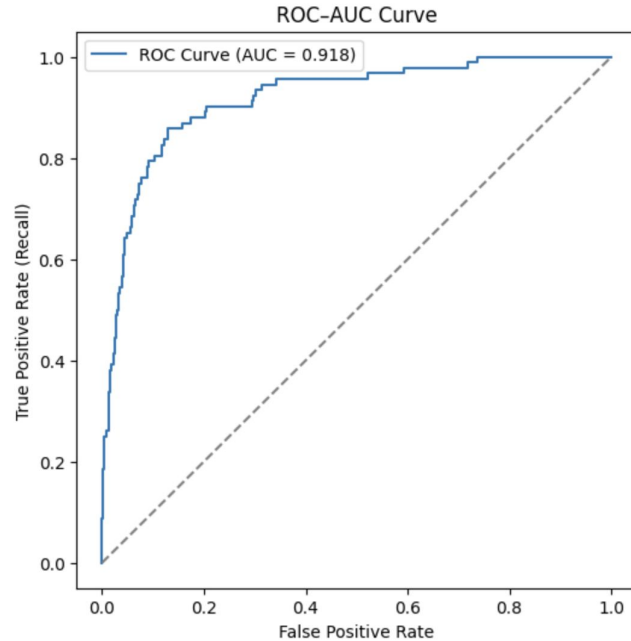
MCC vs F1 Score

	Predicted: 0	Predicted: 1
True: 0	TN = 3	FP = 2
True: 1	FN = 4	TP = 91

The F1-score depends on which class is labeled as positive or negative, but MCC does not.

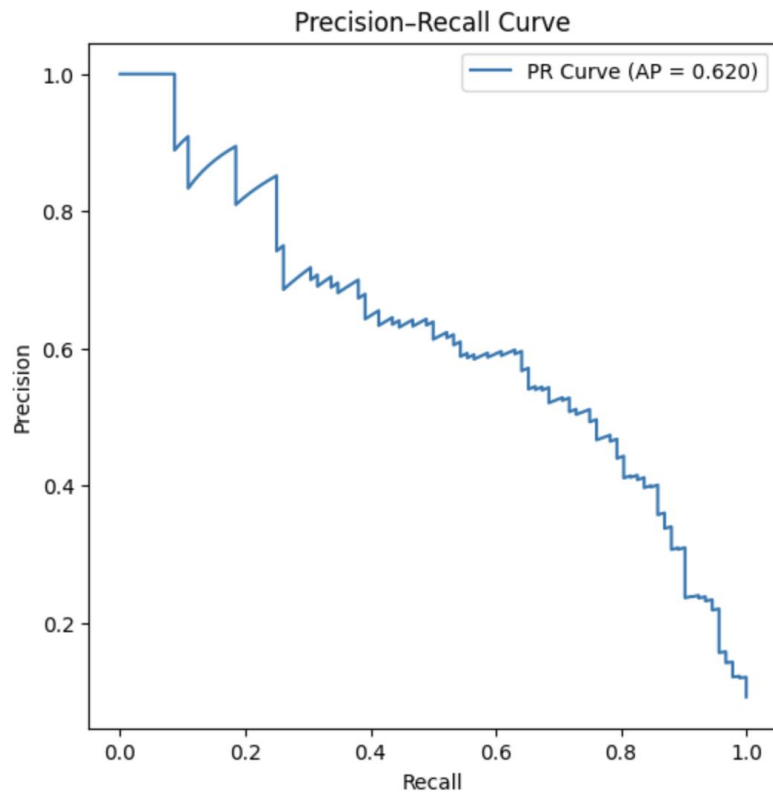
ROC (Receiver Operating Characteristic)-AUC Curve

$$\text{True Positive Rate} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$



$$\text{False Positive Rate} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

Precision/Recall Curve



When to use ROC/AUC

1. **Balanced datasets:** When both classes (positive and negative) are roughly equal.
2. **Symmetric errors:** When false positives (FP) and false negatives (FN) are equally costly.

Example:

1. Topic classification where both categories have enough examples.
2. Sentiment classification (positive vs negative) with balanced training data.

When to use Precision/Recall

1. **Imbalanced datasets:** When positives are rare compared to negatives.
2. **Positive-class focus:** When the key question is "How well does the model find and correctly identify positives?"
3. **Asymmetric errors:** When false positives (FP) and false negatives (FN) have very different costs.

Example:

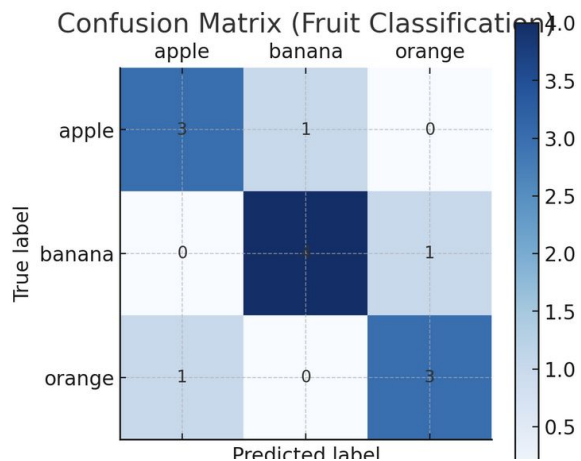
1. Spam filtering (only a small % of emails are spam).
2. Toxic comment detection (most comments are normal, few are toxic).
3. Rare disease detection in medical text.

LogLoss

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Text classification (multi-label classification)

Micro, Macro, Weighted



		TP	FP
1	apple	3	1
2	banana	4	1
3	orange	3	1
4	TOTAL	10	3

$$\text{Micro Precision} = \frac{\sum_i TP_i}{\sum_i (TP_i + FP_i)}$$

$$\text{Macro Precision} = \frac{1}{N} \sum_{i=1}^N \text{Precision}_i$$

$$\text{Weighted Precision} = \frac{\sum_{i=1}^N (\text{support}_i \cdot \text{Precision}_i)}{\sum_{i=1}^N \text{support}_i}$$

Micro, Macro, Weighted

1. **Micro-averaged:** all samples equally contribute to the average
2. **Macro-averaged:** all classes equally contribute to the average
3. **Weighted-average:** each classes's contribution to the average is weighted by its size

Micro, Macro, Weighted

You run a restaurant and customers order 3 types of pizza:

- **Cheese Pizza (100 orders)**
- **Pepperoni Pizza (10 orders)**
- **Vegan Pizza (5 orders)**

Your chef is great at making **Cheese pizzas (90% correct)**, decent at **Pepperoni (60%)**, but often messes up **Vegan (40%)**.

◆ **Micro Average → *Every Slice Counts***

- Look at all slices of pizza sold, no matter the type.
- Since **Cheese pizza is 100 orders**, it dominates the score.
- Result looks **high** because most pizzas are cheese, and those are made well.

👉 Intuition: "How good is my restaurant *overall*, considering every pizza equally?"

Micro, Macro, Weighted

◆ Macro Average → *Every Type Counts Equally*

- Pretend **each pizza type is equally important**, even if only 5 people ordered Vegan.
- You take the average quality of Cheese (90%), Pepperoni (60%), Vegan (40%).
- The **bad Vegan pizzas hurt a lot**, even though they're rare.

👉 Intuition: "How fair am I across all pizza types, giving equal importance to each?"

◆ Weighted Average → *Bigger Orders Count More, but Small Ones Still Count*

- Now you weigh each pizza type by how many were sold.
- Cheese still matters more (100 orders), but Vegan isn't completely ignored (5 orders).
- Score is between micro and macro.

👉 Intuition: "How good am I overall, but adjusting for the fact that some pizzas are more popular?"

Information Retrieval (IR)

Information Retrieval

Task: Given a **query**, retrieve the most relevant documents/items from a large collection.

Examples:

1. Web search (Google returning relevant pages for “machine translation BLEU score”)
2. Legal/medical document search
3. RAG systems (retrieving documents for LLM context)
4. Recommender systems (retrieving items for users)

The **goal** is to rank results so that the **most relevant** documents are at the top.

Information Retrieval

$$Precision = \frac{\text{Relevant Retrieved}}{\text{Total Retrieved}}$$

$$Recall = \frac{\text{Relevant Retrieved}}{\text{Total Relevant in Collection}}$$

Information Retrieval

$$Precision = \frac{\text{Relevant Retrieved}}{\text{Total Retrieved}}$$

$$Recall = \frac{\text{Relevant Retrieved}}{\text{Total Relevant in Collection}}$$

$$P@k = \frac{\text{Relevant in top-k}}{k}$$

$$R@k = \frac{\text{Relevant in top-k}}{\text{Total Relevant}}$$

Evaluation of Generation Tasks

Evaluation of Generation Tasks

Why Generation Tasks Are Hard to Evaluate?

Evaluation of Generation Tasks

Why Generation Tasks Are Hard to Evaluate?

1. Unlike classification (where the answer is yes/no, class A/B/C), **generation has many valid answers**.
 - a. Example: “Translate ‘Я люблю Львів’ into English.”
 - i. Possible outputs:
 1. “I love Kyiv”
 2. “I like Kyiv”
 3. “I am fond of Kyiv”
 - b. All are correct, but an automatic metric might only reward the first.
2. This creates the **problem of variability and subjectivity**.

BLEU Score

BLEU measures how close a machine-generated text (candidate) is to one or more human reference texts.

It does this by checking **n-gram** overlaps (unigrams, bigrams, trigrams, ...) and adjusting with a **brevity penalty**.

BLEU Score - step 1

Suppose the sentence reference (provided by human) is: **"the cat is on the mat"**

And the machine translation (candidate) is: **"the cat the cat on the mat"**

BLEU Score - step 2 (Unigram Precision (1-gram))

We count how many words in the candidate also appear in the reference.

- Candidate unigrams: the, cat, the, cat, on, the, mat
- Reference unigrams: the, cat, is, on, the, mat

Count overlap (with clipping: we cannot count a word more times than it appears in the reference):

- the: appears 3 times in candidate, but only 2 in reference → count = 2
- cat: appears 2 times in candidate, but only 1 in reference → count = 1
- on: appears once, reference also once → count = 1
- mat: appears once, reference also once → count = 1
- Total matched = 2+1+1+1 = 5

$$P_1 = \frac{5}{7} \approx 0.71$$

BLEU Score - step 2 (Bigram Precision (2-gram))

Now check word pairs.

- Candidate bigrams: the cat, cat the, the cat, cat on, on the, the mat
- Reference bigrams: the cat, cat is, is on, on the, the mat

Match (with clipping):

- the cat → in reference (count once)
- on the → in reference
- the mat → in reference

Matches = 3

Total bigrams in candidate = 6

$$P_2 = \frac{3}{6} = 0.5$$

BLEU Score - step 3 (Higher n-grams)

We can continue with trigrams, 4-grams, etc. Typically BLEU uses up to **4-grams**.

BLEU Score - step 4 (Brevity Penalty (BP))

BLEU penalizes very short translations (to avoid cheating by outputting just a few common words).

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

c = candidate length (7 words)

r = reference length (6 words)

Here, candidate is longer $\rightarrow$ $BP = 1$ (no penalty).

BLEU Score - step 5 (BLEU Score Calculation)

BLEU combines precisions with a geometric mean:

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log P_n \right)$$

$$\begin{aligned} BLEU &= 1 \cdot \exp (0.5 \cdot \log(0.71) + 0.5 \cdot \log(0.5)) \\ &= \exp (0.5 \cdot (-0.34) + 0.5 \cdot (-0.69)) \\ &= \exp(-0.515) \approx 0.60 \end{aligned}$$

BLEU Score Examples

Ref: "the cat is on the mat"

Cand: "the cat is on the mat"

All n-grams match → BLEU = 1.0 (100%).

Ref: "the cat is on the mat"

Cand: "the cat on mat"

Fewer n-gram matches, shorter → BLEU much lower (~0.35).

Ref: "the cat is on the mat"

Cand: "on the mat is the cat"

Most unigrams overlap, but bigrams/trigrams don't → BLEU penalizes order → ~0.5.

Pros of BLEU Score

1. BLEU is easy to understand: it checks n-gram overlaps between machine output and human reference translations.
2. Doesn't require human annotators once references are prepared.
3. BLEU only looks at word sequences, not grammar rules or language structure.
4. When averaging over many sentences (e.g., test set of 1k+ sentences), BLEU correlates well with human evaluations of translation quality.

Cons of BLEU Score

1. BLEU is unreliable for individual sentences; it works better at the corpus level.
Example: Ref: **"the cat is on the mat"**, Cand: **"on the mat is a cat"** Meaning is the same, but BLEU score may be low due to order differences.
2. BLEU requires exact word matches. Example: Ref: **"the child is happy"**, Cand: **"the kid is joyful"**. Semantically correct, but BLEU score will be very low.
3. BLEU only checks n-gram overlap, not whether the translation is grammatical or natural. Example: **"the mat is on cat the"** could still get a decent unigram match.
4. Short translations get punished heavily, sometimes unfairly. Example: Ref: **"I am going home now"**, Cand: **"going home"**. Translation might still be useful, but BLEU score drops sharply.

ROUGE Score

ROUGE is a family of metrics for evaluating text generation, especially summarization.

Unlike BLEU (which is precision-oriented), ROUGE is recall-oriented → it measures how much of the reference is covered by the candidate.

BLEU asks: “How much of what the system wrote is correct?”

ROUGE asks: “How much of the reference did the system capture?”

Main Variants of ROUGE

1. ROUGE-N: Overlap of n-grams (e.g., ROUGE-1 = unigrams, ROUGE-2 = bigrams).
2. ROUGE-L: Longest Common Subsequence (order-sensitive).
3. ROUGE-S (Skip-bigram): In-order word pairs, not necessarily consecutive.
4. ROUGE-SU: Skip-bigrams + unigrams (most common in summarization).

ROUGE

Reference Summary (human): **"the cat is on the mat"**, Candidate Summary (system): **"the cat sat on mat"**.

ROUGE-1 (Unigram Recall)

Reference unigrams: the, cat, is, on, the, mat

Candidate unigrams: the, cat, sat, on, mat

Matches: the, cat, on, mat = 4

Reference total = 6

$$ROUGE_1 = \frac{4}{6} = 0.67$$

ROUGE-2 (Bigram Recall)

Reference bigrams: the cat, cat is, is on, on the, the mat

Candidate bigrams: the cat, cat sat, sat on, on mat

Matches: the cat, on mat = 2

Reference total = 5

$$ROUGE_2 = \frac{2}{5} = 0.40$$

ROUGE-L (Longest Common Subsequence)

Longest common subsequence: the cat on mat (length = 4)

Reference length = 6

$$ROUGE_L = \frac{4}{6} = 0.67$$

ROUGE-S (Skip-Bigram Recall)

Reference skip-bigrams (15 total, e.g., (the, cat), (the, on), (cat, mat) ...)

Candidate skip-bigrams (10 total, e.g., (the, cat), (the, on), (cat, mat) ...)

Overlap = 6

$$ROUGE_S = \frac{6}{15} = 0.40$$

Pros of ROUGE

1. Captures content coverage rather than strict precision.
2. Flexible Variants
 - a. ROUGE-N for word-level overlap.
 - b. ROUGE-L for order-sensitive overlap.
 - c. ROUGE-S for flexible word order.

Cons of ROUGE

1. Doesn't handle synonyms or paraphrasing.
2. Since it's recall-focused, longer outputs tend to get higher scores.

METEOR Score

METEOR = **Metric for Evaluation of Translation with Explicit ORdering** (Banerjee & Lavie, 2005). It was designed as an improvement over **BLEU** and **ROUGE**, addressing their weaknesses.

Main idea:

- Match candidate translation to reference using **not just exact words**, but also **stems, synonyms, and paraphrases**.
- Combines **precision + recall** into one score (BLEU is precision-heavy, ROUGE is recall-heavy).
- Includes a **penalty for word-order differences**.

METEOR Score - Step 1

Reference Sentence: **"the cat is on the mat"**, Candidate Sentence: **"a cat sits on mat"**

METEOR Score - Step 2 (Matching Beyond Exact Words)

METEOR tries multiple match levels:

- **Exact match** → words identical
- **Stem match** → **sits** vs **sit**
- **Synonym match** (via WordNet or custom dictionaries) → a vs the might be tolerated in some contexts
- **Paraphrase match** (phrase-level)

Matches found here:

- **cat** (exact)
- **sits** ↔ **is** (stem/synonym mapping)
on (exact)
- **mat** (exact)
- **a** ↔ **the** (article synonym)

So we get **5 matches out of 6 reference words**.

METEOR Score - Step 3 (Precision and Recall)

Precision = (matches / candidate length)

Recall = (matches / reference length)

Candidate = 5 words, Reference = 6 words, Matches = 5

$$P = \frac{5}{5} = 1.0$$

$$R = \frac{5}{6} \approx 0.83$$

METEOR Score - Step 4 (Harmonic Mean (F-score))

$$F_{mean} = \frac{10 \cdot P \cdot R}{R + 9P}$$

$$F_{mean} = \frac{10 \cdot 1.0 \cdot 0.83}{0.83 + 9 \cdot 1.0} = \frac{8.3}{9.83} \approx 0.84$$

METEOR Score - Step 5 (Penalty For Fragmentation)

$$Penalty = \gamma \cdot \left(\frac{ch}{m} \right)^{\beta}$$

ch = number of **chunks** (groups of consecutive matched words in order)

m = number of **matches** (total matched words)

γ, β = parameters (default: $\gamma = 0.5, \beta = 3$)

METEOR Score - Step 5 (Penalty For Fragmentation)

Reference:

"the cat is on the mat"

Candidate:

"the cat on the mat is"

- Matches = 6
- Chunks: the cat + on the mat + is = **3 chunks**

$$Penalty = 0.5 \cdot \left(\frac{3}{6}\right)^3 = 0.5 \cdot (0.5^3) = 0.5 \cdot 0.125 = 0.0625$$

- Small penalty → score slightly reduced.

Meteor Score - Step 6

$$Score = F_{mean} \cdot (1 - Penalty)$$

Pros of Meteor Score

1. Understands Synonyms & Morphology
2. Balances Precision and Recall
3. Captures Word Order

Cons of Meteor Score

1. Needs stemming, synonym dictionaries (WordNet, etc.), so evaluation takes longer than BLEU.
2. Requires linguistic resources (stemmers, synonym sets).
Works best for English; harder for low-resource languages.
3. Doesn't use embeddings or deep semantics, so paraphrases outside its synonym lists get missed.
4. BLEU remains the “default” for MT benchmarks, so METEOR is less used despite being better.

Bert Score

BERTScore (Zhang et al., 2020) is a metric that evaluates text generation (translation, summarization, etc.) using semantic similarity of embeddings instead of raw word overlap.

Main idea:

1. Represent words (or tokens) with contextual embeddings (from a pretrained model like BERT, RoBERTa, or multilingual-BERT).
2. Compare the candidate and reference by measuring cosine similarity between their token embeddings.
3. Output a precision, recall, and F1 score like in information retrieval.

Bert Score - Step 1

Reference sentence: **"the cat is on the mat"**, Candidate sentence: **"the kitty sits on a rug"**

Bert Score - Step 2 (Embedding with BERT)

Each token is passed through BERT (or another large LM).

Example (simplified embeddings shown as vectors):

- Ref: [the, cat, is, on, the, mat] → [[0.1,...], [0.9,...], ...]
- Cand: [the, kitty, sits, on, a, rug] → [[0.1,...], [0.88,...], ...]

Bert Score - Step 3 (Pairwise Cosine Similarity)

For each candidate token, find its **most similar reference token**:

- "the" ↔ "the" → high similarity (≈ 1.0)
- "kitty" ↔ "cat" → high similarity (≈ 0.92)
- "sits" ↔ "is" → medium similarity (≈ 0.75)
- "on" ↔ "on" → high similarity (≈ 1.0)
- "a" ↔ "the" → medium similarity (≈ 0.7)
- "rug" ↔ "mat" → high similarity (≈ 0.89)

Bert Score - Step 4 (Pairwise Cosine Similarity)

Precision: how well candidate tokens are represented in the reference.
(average max similarity over candidate tokens)

Recall: how well reference tokens are covered by candidate tokens.
(average max similarity over reference tokens)

Bert Score - Step 5 (F1 Score)

$$BERTScore = \frac{2 \cdot P \cdot R}{P + R}$$

Pros of BertScore

1. Semantic Awareness
2. With multilingual-BERT, works across many languages (useful for Ukrainian, Polish, etc.).
3. Handles Paraphrasing

Cons of BertScore

1. Computationally Expensive
2. Model Dependence
3. Bias Toward Embedding Space
4. Not Interpretable

Summary

1. BLEU = “Did the candidate use the same words?”
2. ROUGE = “Did the candidate cover the same content?”
3. METEOR = “Did the candidate match words/synonyms in order?”
4. BERTScore = “Does the candidate mean the same thing?”

COMET

COMET is a **learned evaluation metric** for MT.

Instead of just counting n-gram overlaps (BLEU/ROUGE) or computing raw embedding similarities (BERTScore), COMET uses a **neural model trained on human judgments of translation quality**.

It predicts how good a candidate translation is by comparing it to the **source** and the **reference**.

Model-as-a-Judge

Instead of using an automatic metric like **BLEU**, **ROUGE**, **METEOR**, **BERTScore**, we ask a **LLM** to evaluate the quality of system outputs directly.

For example:

- Task: *translation evaluation*
- Reference: "the cat is on the mat"
- Candidate: "the kitty is sitting on the rug"
- Instead of computing n-gram overlaps, you prompt a model like GPT-4:
"On a scale of 1–10, how well does the candidate convey the same meaning as the reference? Explain briefly."

Pros of Model-as-a-Judge

1. Semantic Understanding
2. Can evaluate not just adequacy, but also fluency, style, faithfulness, factuality.
3. Model can output why a translation is good/bad — BLEU cannot.

Cons of Model-as-a-Judge

1. Two runs of the same evaluation may produce different judgments unless carefully prompted (though randomness can be reduced with temperature=0).
2. Running GPT-4 or similar as a judge is far more expensive than BLEU/ROUGE.

Human Evaluation

1. Types of human annotation?
2. Pros/Cons of human annotation?

Taking into account everything that we have discussed, what strategy would you chose to evaluate the LLM you trained?

Takeaways

1. There is no “one-size-fits-all” metric
2. Understand your data
3. Good benchmarks worth a million
4. Align with business objectives
5. Use common sense and involve domain experts
6. Remember: The evaluation metric is the proxy for the real world.